



Universidad Experimental De Guayana

Vicerrectorado Académico

Coordinación General De Pregrado

Proyecto De Carrera: Ingeniería En Informática

Asignatura: Lenguajes y Compiladores

Análisis Léxico

Profesor:

Márquez Félix

Alumnos:

Aguilera Rhixeidys V-30.851.503

Albornoz Ronniel V- 24.889.173

Bravo Yvanna V- 28.530.717

Curbata Emilio V- 28.214.578

Ciudad Guayana, julio de 2026

Resumen Académico

El presente informe detalla el diseño, la fundamentación teórica y la implementación práctica de analizadores léxicos, abarcando diferentes enfoques metodológicos en la fase inicial de la compilación. Se inicia con el estudio de los autómatas de pila y su superioridad computacional sobre los autómatas finitos tradicionales para el reconocimiento de gramáticas de libre contexto y la validación de estructuras jerárquicas. Posteriormente, se exponen dos estrategias prácticas de tokenización: la construcción manual de un lexer tolerante a fallos para archivos Docker mediante expresiones regulares, y el desarrollo automatizado de un analizador léxico para un subconjunto del lenguaje Rust utilizando la herramienta de metacompilación Flex. Finalmente, el documento reflexiona sobre la criticidad del análisis léxico en el ámbito de la ciberseguridad, ilustrando cómo estas tecnologías garantizan la fiabilidad en la tokenización de firmas de detección en lenguajes como YARA.

Introducción

El análisis léxico constituye la primera fase fundamental en el proceso de compilación, actuando como el puente indispensable entre el código fuente escrito por el programador y la tokenización lógica necesaria para el posterior procesamiento computacional. En este contexto, la teoría de autómatas proporciona el sustento matemático riguroso para el diseño de estas herramientas, permitiendo clasificar y procesar distintos tipos de lenguajes formales mediante el reconocimiento de patrones. Mientras que los autómatas finitos deterministas (AFD) y no deterministas (AFND) son suficientes para agrupar caracteres en *tokens* basados en lenguajes regulares, la evaluación de estructuras anidadas más complejas exige el uso de modelos computacionales superiores, tales como los autómatas de pila, los cuales incorporan una memoria auxiliar de tipo LIFO para posibilitar el análisis de gramáticas de libre contexto. Comprender estas bases teóricas y metodológicas es de vital importancia para los profesionales de la Ingeniería en Informática, ya que fundamenta el funcionamiento interno de las arquitecturas de software y establece los cimientos lógicos ineludibles para la validación sintáctica.

El presente informe tiene como propósito central explorar tanto la teoría subyacente como la implementación empírica del análisis léxico, desarrollando cuatro actividades prácticas que consolidan los objetivos de la asignatura de Lenguajes y Compiladores. En primer lugar, se abordan las definiciones conceptuales y las aplicaciones algorítmicas de los autómatas de pila, contrastando su poder y relevancia frente a los autómatas convencionales. Seguidamente, se documenta la programación manual de un analizador léxico desarrollado en Python, diseñado específicamente para validar de manera ininterrumpida los componentes de un archivo *Dockerfile* mediante un esquema de control de errores de "estado fallido". A continuación, el trabajo profundiza en el paradigma de la metacompilación mediante la integración de la herramienta Flex, detallando la estructuración de reglas y la generación automatizada en lenguaje C de un lexer para un subconjunto representativo del lenguaje Rust. Finalmente, el documento cierra con un análisis transversal sobre la importancia crítica del análisis léxico en la seguridad informática, demostrando su aplicabilidad en la validación de reglas operativas YARA para la detección temprana de anomalías y *malware*.

1. Definición y Ejemplos de Autómatas de Pila

1.1 Definición Formal de un Autómata de Pila (AP)

Un Autómata de Pila (AP, o Pushdown Automaton en inglés) es un modelo computacional teórico que amplía la capacidad de un autómata finito al incorporar una memoria auxiliar estructurada como una pila (LIFO: Last In, First Out). Mientras que los autómatas finitos solo pueden basar sus decisiones en el estado actual, el autómata de pila puede almacenar y recuperar una cantidad ilimitada de símbolos, lo que le permite "recordar" elementos procesados previamente.

Matemáticamente, un Autómata de Pila se define como una séptupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, donde:

- Q : Es un conjunto finito de estados (similar a un AFD).
- Σ : Es el alfabeto de entrada (conjunto finito de símbolos que puede leer la máquina).
- Γ : Es el alfabeto de la pila (símbolos que pueden almacenarse en la memoria auxiliar).
- δ : Es la función de transición. A diferencia de un AFD clásico, esta función toma tres argumentos: el estado actual, el símbolo de entrada leído ($\epsilon \in \Sigma$ para transiciones vacías) y el símbolo en el tope de la pila. Retorna un nuevo estado y una cadena de símbolos para reemplazar el tope de la pila.
- $q_0 \in Q$: Es el estado inicial.
- $Z_0 \in \Gamma$: Es el símbolo inicial de la pila (marca el fondo de la misma).
- $F \subseteq Q$: Es el conjunto de estados de aceptación o finales.

1.2 Ejemplos Prácticos de Autómatas de Pila

A continuación, se presentan tres ejemplos de complejidad progresiva que ilustran la capacidad de los AP para procesar lenguajes que escapan a los autómatas finitos:

- **Ejemplo 1 (Básico): Reconocimiento del lenguaje $L = \{0^n 1^n | n \geq 1\}$**
 - **Descripción:** Este lenguaje exige que la cadena tenga una cantidad n de ceros seguida exactamente por la misma cantidad n de unos.
 - **Funcionamiento del AP:**
 1. Por cada "0" que el autómata lee en la cadena de entrada, **apila** un símbolo (por ejemplo, 'X') en la memoria.
 2. Cuando comienza a leer los "1", por cada "1" que lee, **desapila** una 'X'.
 3. Si al terminar de leer la cadena de entrada la pila queda vacía (llegando al símbolo Z_0), la cadena es aceptada. Si sobran 'X' en la pila o faltan 'X' para desapilar, la rechaza.
- **Ejemplo 2 (Intermedio): Verificación de paréntesis balanceados**
 - **Descripción:** Un problema clásico en la computación es verificar que una expresión matemática o bloque de código tenga los paréntesis cerrados correctamente (ej. $()()$ o $()()$).
 - **Funcionamiento del AP:**
 1. Al leer un paréntesis de apertura (, el autómata **apila** un marcador.
 2. Al leer un paréntesis de cierre), el autómata exige **desapilar** un marcador. Si intenta desapilar pero la pila está vacía, rechaza la cadena inmediatamente (ej. $()$)).
 3. Al finalizar la lectura de la expresión, si la pila está en su estado inicial, los paréntesis están balanceados y acepta.
- **Ejemplo 3 (Avanzado): Palíndromos con marcador central $L = \{wcw^R | w \in (a,b)^*\}$**
 - **Descripción:** Reconocer cadenas que se leen igual de izquierda a derecha que de derecha a izquierda, separadas por un carácter central 'c' (ejemplo: abb c bba).
 - **Funcionamiento del AP:**
 1. Lee la primera parte de la cadena (w) y **apila** cada símbolo ('a' o 'b').

2. Al encontrar el marcador central 'c', el autómata cambia de estado y deja de apilar.
3. Para la segunda parte (w^R), compara el símbolo de entrada con el símbolo que **desapila** del tope. Si todos coinciden hasta vaciar la entrada y la pila, la acepta.

1.3 Importancia y Poder Computacional: AP frente a AFD y AFND

Para comprender la jerarquía de poder, debemos contrastar el AP con los modelos más simples utilizados en el análisis léxico:

- **Autómatas Finitos (AFD y AFND):** Un autómata finito determinístico (AFD) o no determinístico (AFND) posee una memoria estrictamente limitada a su estado actual. Son suficientes para reconocer componentes léxicos (tokens) de un lenguaje formal porque estos obedecen a lenguajes regulares (la base de la jerarquía de Chomsky). Sin embargo, carecen de un mecanismo para "contar" de forma ilimitada o validar anidamientos profundos.
- **Poder adicional del Autómata de Pila:** El autómata de pila es estrictamente superior en poder computacional a los AFD y AFND. Gracias a su memoria LIFO de capacidad infinita, rompe la barrera de los lenguajes regulares y es capaz de reconocer lenguajes más complejos denominados Gramáticas de Libre Contexto (GLC). Mientras un AFD puede identificar perfectamente que `while` es una palabra reservada o que `10` es un número entero, es matemáticamente incapaz de verificar si hay 50 llaves de apertura `{` y exactamente 50 llaves de cierre `}` en un bloque de código; tarea que el AP resuelve de forma trivial.

En el contexto del diseño de lenguajes y compiladores, la utilidad de los autómatas de pila radica en que constituyen el "motor teórico" del **Análisis Sintáctico** (o Parser).

Mientras que el analizador léxico (*lexer*) se apoya en expresiones regulares y Autómatas Finitos para agrupar caracteres en *tokens* (como identificadores u operadores), estas estructuras lineales no pueden validar la arquitectura jerárquica del código. El autómata de pila permite a los compiladores evaluar el árbol de derivación de las Gramáticas de Libre Contexto (GLC), haciendo posible que herramientas más avanzadas comprendan estructuras anidadas complejas, precedencia de operadores lógicos y bloques

de control en lenguajes modernos. Sin el autómata de pila, la compilación de la sintaxis estructural del software que desarrollamos hoy en día sería computacionalmente imposible.

2. Construcción de un lexer para la verificación de archivos docker mediante expresiones regulares.

2.1 Componentes léxicos del lenguaje (Dockerfile)

Token	Descripción	Ejemplo
COMMENT	Línea que inicia con #	# comentario
INSTRUCCION	Palabra reservada	FROM, RUN, COPY, ENV...
FLAG	Bandera de instrucción	--from=builder
ENV_VAR	clave=valor (ENV/ARG)	APP_ENV=production
NUMERO	Puerto o número	8080/tcp , 1000
STRING	Cadena entre comillas	"daemon off;"
JSON_ARRAY	Arreglo forma exec	["python","app.py"]
IMAGEN_TAG	Imagen base + tag/digest	python:3.12-slim
RUTA	Ruta / identificador	/app

2.2 Construcción del lexer (resumen)

- Se define un token por cada componente léxico, con su expresión regular.
- El orden importa: reglas específicas (INSTRUCCION, FLAG, ENV_VAR, NUMERO, STRING, JSON_ARRAY) van antes que las genéricas (IMAGEN_TAG, RUTA).
- Todos los patrones se combinan en una regex maestra con grupos con nombre: (?P<TOKEN>patrón).
- Se recorre la entrada con `re.finditer`, identificando cada coincidencia con `mo.lastgroup`.
- Ante un carácter no reconocido (MISMATCH) se reporta línea/columna y el análisis continúa (no se detiene en el primer error).

```
tokens = [  
    ("INSTRUCCION", r"\b(?:FROM|RUN|CMD|COPY|ENV|EXPOSE|...)\b"),  
    ("FLAG", r"--[a-zA-Z][a-zA-Z0-9_-]*(=[^\s]+)?"),  
    ("ENV_VAR", r"[A-Za-z_][A-Za-z0-9_-]*=[^\s]+"),  
    ("NUMERO", r"\b\d{1,5}(/(tcp|udp))?\b"),
```

```

("IMAGEN_TAG", r"[a-zA-Z0-9_.\-/+](:[a-zA-Z0-9_.\-/+)?(@sha256:[a-fA-F0-9]{6,64})?"),
("MISMATCH", r"."),
]

regex_maestra = "|".join(f"(?P<{n}>{p})" for n, p in tokens)
for mo in re.finditer(regex_maestra, texto, re.MULTILINE):
    tipo, valor = mo.lastgroup, mo.group(mo.lastgroup)
    if tipo == "MISMATCH": registrar_error(valor, linea, col); continue
    yield (tipo, valor, linea, col)

```

2.3 Ejecución

- `python3 lexer_docker.py <archivo_dockerfile>`
- `./dist/lexer_docker <archivo_dockerfile>` (ejecutable, sin Python instalado)

2.4 Ejemplos de ejecución

Ejemplo 1 — Dockerfile correcto

```

FROM python:3.12-slim          ENV PYTHONUNBUFFERED=1
WORKDIR /app                  EXPOSE 8080/tcp
CMD ["python", "app.py"]

→ ('INSTRUCCION', 'FROM', 2, 0)  ('IMAGEN_TAG', 'python:3.12-slim', 2, 5)
→ ('NUMERO', '8080/tcp', 9, 7)   ('JSON_ARRAY', '["python", "app.py"]', 10, 4)
→ Total de tokens: 23 | 0 errores

```

Ejemplo 2 — Multi-stage, flags y digest

```

FROM golang:1.22 AS builder    FROM alpine@sha256:c5b1261d...
COPY --from=builder /src/go.mod . USER 1000

→ ('FLAG', '--from=builder', 5, 5)
→ ('IMAGEN_TAG', 'alpine@sha256:c5b1261d...', 8, 5)
→ ('NUMERO', '1000', 11, 5)
→ Total de tokens: 30 | 0 errores

```

Ejemplo 3 — Con errores léxicos

```

RUN echo "deploy % ok" @@ true

→ [ERROR LÉXICO] '@' en línea 8, columna 23
→ [ERROR LÉXICO] '@' en línea 8, columna 24
→ El '%' dentro de la cadena NO es error (es parte del STRING)
→ El análisis continúa hasta el final del archivo | Total tokens: 17

```


El Dockerfile se comporta como un lenguaje regular: un conjunto de expresiones regulares (equivalentes a un AFD) basta para tokenizarlo por completo. El manejo con "estado fallido" permite reportar todos los errores léxicos en una sola pasada sin detener el análisis. Validar el orden correcto de las instrucciones queda para una fase posterior de análisis sintáctico.

3. Definición de un lenguaje L subconjunto de lenguaje Rust, construcción de un lexer para L utilizando metacompilador.

L es un subconjunto reducido de Rust orientado a expresar declaraciones de variables, funciones simples, estructuras de control básicas (if/else) y la macro de impresión println!. El propósito de L no es ser un lenguaje completo, sino ofrecer un conjunto de componentes léxicos representativos suficiente para ejercitar la construcción de un analizador léxico mediante metacompilador.

- **Palabras reservadas**

let, mut, fn, if, else, while, for, return, true, false.

- **Tipos de datos**

i32 (entero de 32 bits), f64 (flotante de doble precisión), bool (booleano), String (cadena).

- **Macro soportada**

println! — macro de impresión por consola, reconocida como un único componente léxico.

- **Literales**

- Enteros: secuencia de uno o más dígitos, ej. 10, 25.
- Flotantes: dígitos, punto decimal, dígitos, ej. 3.14.
- Cadenas: texto delimitado por comillas dobles, ej. "x es mayor que 5".

- **Operadores**

Aritméticos: + - * / · Relacionales: == != >= <= > < · Asignación: =

- **Puntuadores**

{ } () ; : ,

- **Identificadores**

Definidos por la expresión regular `[a-zA-Z_][a-zA-Z0-9_]*`, es decir, deben iniciar con una letra o guion bajo y pueden continuar con letras, dígitos o guion bajo. La forma de especificar identificadores mediante una expresión regular de este tipo es el patrón estándar en la literatura de compiladores (*Aho, Lam, Sethi & Ullman, 2007, cap. 3, sec. 3.3, pp. 109-135*).

- **Comentarios**

Comentarios de línea iniciados con `//`; todo el contenido hasta el fin de línea es ignorado por el lexer.

- **Manejo de errores léxicos**

Cualquier carácter que no calce con ninguna de las reglas anteriores es reportado como error léxico, indicando el número de línea y el carácter no reconocido, sin detener la ejecución del analizador. Este diseño seudo con "estados fallidos" que reinician el autómata en el estado inicial, en vez de detener la ejecución al primer error, corresponde a la estrategia de recuperación de errores léxicos discutida en el material de la asignatura (*Márquez, 2026, p. 3*).

2.1 Tabla resumen de tokens de L

Token	Patrón / lexema de ejemplo	Descripción
KW_LET, KW_MUT, KW_FN, KW_IF, KW_ELSE, KW_WHILE, KW_FOR, KW_RETURN, KW_TRUE, KW_FALSE	let, mut, fn, if...	Palabras reservadas del lenguaje
TIPO_I32, TIPO_F64, TIPO_BOOL, TIPO_STRING	i32, f64, bool, String	Tipos de datos
MACRO_PRINTLN	println!	Macro de impresión

LIT_ENTERO	10, 25	Literal numérico entero
LIT_FLOTANTE	3.14, 1.0	Literal numérico flotante
LIT_CADENA	"texto"	Literal de cadena
OP_SUMA, OP_RESTA, OP_MULT, OP_DIV	+ - * /	Operadores aritméticos
OP_IGUAL, OP_DISTINTO, OP_MAYOR_IGU, OP_MENOR_IGU, OP_MAYOR, OP_MENOR	== != >= <= > <	Operadores relacionales
OP_ASIGNACION	=	Operador de asignación
LLAVE_ABR, LLAVE_CIE, PAREN_ABR, PAREN_CIE	{ } ()	Puntuadores de bloque/agrupación
PUNTO_COMA, DOS_PUNTOS, COMA	; : ,	Puntuadores de separación
IDENTIFICADOR	resultado, suma, x	Nombres de variables y funciones

2.2 El metacompilador seleccionado: Flex

El metacompilador elegido para construir el lexer de L es Flex (Fast Lexical Analyzer Generator), en su versión 2.5.4 distribuida a través del proyecto GnuWin32 para Windows. Flex recibe un archivo de especificación con extensión. l, en el cual se definen expresiones regulares asociadas a acciones en lenguaje C, y genera como salida un archivo fuente en C (lex.yy.c) que implementa el autómata finito determinístico correspondiente y la función `yylex()`, encargada de leer la entrada carácter a carácter y devolver los tokens reconocidos. Esta forma de trabajar —describir patrones y dejar que la herramienta derive el autómata y su tabla de transiciones— es precisamente la alternativa a la construcción manual de un AFD que describe la teoría de autómatas finitos deterministas, definidos formalmente como la tupla $(Q, \Sigma, \delta, q_0, F)$ (Márquez, 2026, p. 5; Aho et al., 2007, cap. 3, sec. 3.6, pp. 152-159).

Se seleccionó Flex por ser la herramienta clásica de referencia para la construcción de analizadores léxicos mediante metacompilación, ampliamente documentada en la literatura sobre construcción de compiladores (*Aho et al., 2007, cap. 3, sec. 3.5 "The Lexical-Analyzer Generator Lex"*; *Levine, 2009, cap. 1*), y por su compatibilidad directa con GCC para la generación del ejecutable final.

2.3 Manual de usuario de Flex

A continuación, se documenta, con palabras propias y basado en la experiencia práctica de instalación y uso, el manual de usuario del metacompilador Flex empleado en esta actividad.

- **¿Qué es Flex?**

Flex es un generador de analizadores léxicos: a partir de un archivo de reglas (extensión. l) que combina expresiones regulares con fragmentos de código C, produce automáticamente el código fuente de un programa lexer capaz de reconocer los patrones descritos. En lugar de programar manualmente el autómata finito y su tabla de transiciones, el desarrollador se concentra en declarar qué patrones existen y qué acción tomar cuando se reconocen (*Levine, 2009, cap. 1, "Introducing Flex and Bison"*).

- **Estructura de un archivo. l**

Todo archivo de especificación Flex se organiza en tres secciones separadas por el delimitador `%%`, estructura documentada en detalle en el capítulo dedicado al uso de Flex (*Levine, 2009, cap. 2, "Using Flex"*):

- Sección de definiciones: declaraciones en C (encabezados, variables globales) encerradas entre `% {y %}`, además de opciones de Flex como `%option`.
- Sección de reglas: pares patrón-acción. El patrón es una expresión regular y la acción es código C que se ejecuta al reconocer ese patrón.
- Sección de código de usuario: funciones auxiliares en C, incluida la función `main` () y, si aplica, la función `yywrap` ().

```
%{ /* Sección de definiciones: código C, variables globales */ %} %% /* Sección de
reglas: patrón { acción en C } */ %% /* Sección de código de usuario: main(), funciones
auxiliares */
```

- **Variables y funciones clave**

Las siguientes variables y funciones, generadas o expuestas por Flex, están documentadas como referencia estándar de la herramienta (*Levine, 2009, cap. 5, "A Reference for Flex Specifications"*):

- `yytext`: cadena que contiene el lexema reconocido por la regla que se está ejecutando.
- `yylex()`: función generada por Flex que ejecuta el ciclo de análisis léxico completo sobre la entrada.
- `yyin`: puntero a archivo (`FILE*`) que indica el origen de la entrada; si no se asigna, Flex lee de la entrada estándar (`stdin`).
- `yywrap()`: función invocada al llegar al fin de archivo; retornar 1 indica que no hay más entradas que procesar.

- **Flujo de trabajo general con Flex**

- Escribir el archivo de especificación con extensión. `.l`, definiendo los patrones del lenguaje objetivo.
- Ejecutar el comando `flex archivo.l`, lo cual genera el archivo `lex.yy.c`.
- Compilar `lex.yy.c` con un compilador de C (en este caso GCC) para obtener un ejecutable.
- Ejecutar el programa resultante pasando como argumento el archivo fuente que se desea analizar.

- **Buenas prácticas observadas**

Ordenar las reglas de mayor a menor especificidad primero las palabras reservadas exactas y luego el patrón general de identificador es necesario porque, ante dos patrones que calzan con la misma longitud, Flex prioriza la regla declarada primero; este comportamiento de resolución de ambigüedades está documentado tanto a nivel teórico como práctico (*Aho et al., 2007, cap. 3, sec. 3.4 "Recognition of Tokens"; Levine, 2009, cap. 2*).

- Usar `%option noyywrap` (o definir `yywrap()` manualmente) para evitar la dependencia de la librería `libfl`.

- Incluir siempre una regla catch-all (.) al final para capturar y reportar caracteres no reconocidos como error léxico, en lugar de dejar que Flex los ignore silenciosamente, tal como recomienda el material de la asignatura al describir el control de errores en un analizador léxico (Márquez, 2026, p. 3).

2.4 Proceso de creación del lexer para L

El desarrollo del lexer siguió los siguientes pasos, documentados también en el archivo README.md del repositorio del proyecto:

- **Diseño de las reglas léxicas**

Para cada componente léxico de L (ver tabla de la sección 2.10) se definió una expresión regular y una acción asociada. Las acciones imprimen, para cada token reconocido, el número de línea, el nombre del token y el lexema exacto, con el formato:

```
[LINEA N] TOKEN: TIPO_DE_TOKEN | Lexema: valor_reconocido
```

- **Archivo de especificación lexer_rust.l**

El archivo completo de especificación se muestra a continuación (versión abreviada por espacio). Se destaca el uso de un contador de línea global (variable linea) que se incrementa en la regla asociada al salto de línea (\n), y el orden de las reglas: primero las palabras reservadas y tipos, luego macro y literales, después operadores y puntuadores, y finalmente identificadores, comentarios, espacios y el manejador de errores léxicos.

```
%{ #include <stdio.h> int linea = 1; %} %% "let" { printf("[LINEA %d] TOKEN: KW_LET | Lexema: %s\n", linea, yytext); } "mut" { printf("[LINEA %d] TOKEN: KW_MUT | Lexema: %s\n", linea, yytext); } "fn" { printf("[LINEA %d] TOKEN: KW_FN | Lexema: %s\n", linea, yytext); } ... "i32" { printf("[LINEA %d] TOKEN: TIPO_I32 | Lexema: %s\n", linea, yytext); } "println!" { printf("[LINEA %d] TOKEN: MACRO_PRINTLN | Lexema: %s\n", linea, yytext); } [0-9]+\.[0-9]+ { printf("[LINEA %d] TOKEN: LIT_FLOTANTE | Lexema: %s\n", linea, yytext); } [0-9]+ { printf("[LINEA %d] TOKEN: LIT_ENTERO | Lexema: %s\n", linea, yytext); } \"[^\"]*\" { printf("[LINEA %d] TOKEN: LIT_CADENA | Lexema: %s\n", linea, yytext); } "+" { printf("[LINEA %d] TOKEN: OP_SUMA | Lexema: %s\n", linea,
```

```
yytext); } ... [a-zA-Z_][a-zA-Z0-9_]* { printf("[LINEA %d] TOKEN:
IDENTIFICADOR | Lexema: %s\n", linea, yytext); } \V[^\n]* { /* ignorar comentarios
*/ } \n { linea++; } [ \t]+ { /* ignorar espacios */ } . { printf("[LINEA %d]
ERROR LEXICO: caracter no reconocido '%s'\n", linea, yytext); } %% int main(int
argc, char **argv) { if (argc > 1) { FILE *f = fopen(argv[1], "r"); if (!f) {
printf("Error abriendo archivo\n"); return 1; } yyin = f; } yylex(); return 0; } int
yywrap() { return 1; }
```

Nota: el listado anterior muestra una versión abreviada con puntos suspensivos únicamente por espacio; el archivo `lexer_rust.l` real y completo, con las 34 reglas (una por cada token de la tabla 2.10), se encuentra en el repositorio del proyecto, en la ruta `C:\LexerRust\lexer_rust.l`.

- **Programa de prueba en L**

Para validar el lexer se escribió el archivo `programa_prueba.rs`, que ejercita funciones, variables mutables e inmutables, tipos `i32` y `f64`, estructuras condicionales, la macro `println!` y comentarios de línea:

```
fn suma(a: i32, b: i32) -> i32 { let resultado = a + b; return resultado; } fn
main() { let x: i32 = 10; let mut y: f64 = 3.14; if x > 5 { println!("x es
mayor que 5"); } else { y = y + 1.0; } }
```

2.6 Instalación y configuración del entorno (Windows 8.1)

El entorno de desarrollo se configuró sobre Windows 8.1 utilizando las siguientes herramientas:

- Flex 2.5.4, distribuido mediante el proyecto GnuWin32, instalado en `C:\Program Files (x86)\GnuWin32\bin`.
- GCC 10.3.0 (distribución TDM64-GCC), utilizado para compilar el código C generado por Flex.

- Carpeta de trabajo del proyecto: C:\LexerRust\, donde residen el archivo de especificación, el programa de prueba y los ejecutables generados.

- **Pasos de instalación**

1. Descargar e instalar GnuWin32 Flex 2.5.4, aceptando la ruta de instalación por defecto (C:\Program Files (x86)\GnuWin32).
2. Descargar e instalar TDM-GCC 10.3.0 (64 bits), asegurando incluir gcc.exe en el PATH del sistema o agregarlo manualmente.
3. Crear la carpeta de trabajo C:\LexerRust y ubicar allí el archivo lexer_rust.l y el archivo de prueba programa_prueba.rs.
4. Abrir el Símbolo del sistema (cmd) y agregar GnuWin32 al PATH de la sesión actual, ya que la instalación por defecto no siempre registra la ruta de forma permanente:

```
set PATH=%PATH%;C:\Program Files (x86)\GnuWin32\bin
```

5. Verificar la instalación ejecutando flex --version y gcc --version; ambos comandos deben responder con el número de versión sin errores.

2.5 Compilación y ejecución del lexer

Una vez configurado el entorno, la generación del ejecutable del lexer se realizó en tres pasos, ejecutados desde la carpeta C:\LexerRust:

- **Generar el código C a partir de la especificación**

```
flex lexer_rust.l
```

Este comando procesa lexer_rust.l y genera el archivo lex.yy.c, el cual contiene la implementación en C del autómata que reconoce los tokens de L.

- **Compilar el código generado**

```
gcc lex.yy.c -o lexer_rust.exe
```

GCC compila lex.yy.c y produce el ejecutable lexer_rust.exe.

- **Ejecutar el lexer sobre el programa de prueba**

```
lexer_rust.exe programa_prueba.rs
```


El ejecutable recibe como argumento la ruta del archivo fuente a analizar (programa_prueba.rs) y despliega en la consola, línea por línea, cada token reconocido junto con su lexema.

2.6 Pruebas y resultados obtenidos

La ejecución de lexer_rust.exe sobre programa_prueba.rs reconoció correctamente la totalidad de los tokens definidos para L, respetando el número de línea de cada uno. A continuación, se muestra un extracto representativo de la salida obtenida en consola:

```
[LINEA 1] TOKEN: KW_FN | Lexema: fn [LINEA 1] TOKEN: IDENTIFICADOR |  
Lexema: suma [LINEA 1] TOKEN: PAREN_ABR | Lexema: ( [LINEA 1] TOKEN:  
IDENTIFICADOR | Lexema: a [LINEA 1] TOKEN: DOS_PUNTOS | Lexema: :  
[LINEA 1] TOKEN: TIPO_I32 | Lexema: i32 ... [LINEA 9] TOKEN: KW_IF |  
Lexema: if [LINEA 9] TOKEN: IDENTIFICADOR | Lexema: x [LINEA 9] TOKEN:  
OP_MAYOR | Lexema: > [LINEA 9] TOKEN: LIT_ENTERO | Lexema: 5 [LINEA  
10] TOKEN: MACRO_PRINTLN | Lexema: println! [LINEA 10] TOKEN:  
LIT_CADENA | Lexema: "x es mayor que 5" [LINEA 14] TOKEN: LLAVE_CIE |  
Lexema: }
```

La salida completa (capturada directamente de la ejecución en el Símbolo del sistema de Windows) confirma el reconocimiento correcto de palabras reservadas, tipos, literales enteros y flotantes, ¡la macro println! junto con su literal de cadena, todos los operadores y puntuadores, y los identificadores definidos por el usuario, todo asociado correctamente a su número de línea.

No se registraron errores léxicos durante la prueba, dado que programa_prueba.rs fue escrito íntegramente con componentes válidos del lenguaje L. La regla catch-all (.) del archivo lexer_rust.l fue verificada de forma independiente introduciendo caracteres no contemplados por L (por ejemplo, @ o \$), confirmando que el lexer reporta correctamente el error con el formato [LINEA N] ERROR LEXICO: caracter no reconocido.

4. Reflexión y Aplicación de FLEX en Lenguajes de Seguridad Informática

4.1 Reflexión: El papel de los analizadores léxicos en Ciberseguridad

En el área de la seguridad informática, la capacidad de analizar texto de manera rápida, automatizada y sin errores es crítica. Las herramientas de seguridad (como firewalls, sistemas de detección de intrusos y analizadores de malware) procesan constantemente archivos de configuración, reglas de detección y *logs* de red.

Un metacompilador como FLEX resulta ideal en este contexto porque facilita la creación rápida de analizadores léxicos. El análisis léxico es la fase fundamental que transforma una secuencia de caracteres en unidades lógicas llamadas *tokens*. Al aplicar FLEX en ciberseguridad, se puede garantizar que las firmas de detección o las reglas de seguridad introducidas por los analistas estén libres de errores léxicos antes de que el motor de seguridad intente ejecutarlas, previniendo fallos críticos en los sistemas de defensa.

4.2 Lenguajes aplicables en el área

Existen varios lenguajes de dominio específico (DSL) en ciberseguridad donde se podría aplicar un analizador generado por FLEX:

1. **Reglas de Snort / Suricata:** Lenguajes utilizados en Sistemas de Detección de Intrusos (IDS) para definir patrones de tráfico de red malicioso.
2. **Scripts de Zeek (anteriormente Bro):** Un lenguaje de scripting orientado a eventos para el análisis de tráfico de red de alto nivel.
3. **Reglas YARA:** Conocido como el "cuchillo suizo" de los investigadores de malware, es un lenguaje que permite identificar y clasificar muestras de malware basándose en patrones textuales o binarios.

4.3 Presentación del Lenguaje Seleccionado: Reglas YARA

Para este desarrollo, seleccionaremos el lenguaje de Reglas YARA. Su sintaxis es similar a la del lenguaje C y posee una estructura altamente estandarizada que requiere una correcta tokenización para su procesamiento.

- **Ejemplo de una regla YARA básica (Entrada del Lexer):**

```
rule Deteccion_Malware {
strings:
$cadena_sospechosa = "cmd.exe /c"
condition:
$cadena_sospechosa
}
```

4.4 Tokenización del Lenguaje YARA

Para que un programa entienda esta regla, el analizador léxico debe identificar patrones en los caracteres leídos y agruparlos en componentes léxicos (tokens).

A continuación, se presenta la tabla de tokenización de la regla de ejemplo y su respectiva implementación teórica en un archivo FLEX (.l):

Lexema leído	Token Identificado	Descripción
rule	TK_RULE	Palabra reservada que inicia la regla.
Deteccion_Malware	TK_IDENTIFICADOR	Nombre asignado por el usuario a la regla.
{	TK_LLAVE_ABRE	Delimitador de inicio de bloque.
strings:	TK_STRINGS	Palabra reservada para la sección de variables.
\$cadena_sospechosa	TK_VARIABLE	Identificador de variable (siempre inicia con \$).
=	TK_ASIGNACION	Operador de asignación.
"cmd.exe /c"	TK_CADENA_LITERAL	Valor asignado a evaluar.
condition:	TK_CONDITION	Palabra reservada para la sección de evaluación.
}	TK_LLAVE_CIERRA	Delimitador de fin de bloque.

4.5 Implementación Teórica en Metacompilador FLEX (lexer_yara.l): Esta es la estructura del archivo que se utilizaría para definir los patrones (expresiones regulares) de los componentes léxicos asociados al lenguaje YARA.

```
%{
#include <stdio.h>
%}
%option noyywrap
%%
"rule"                { printf("TOKEN: TK_RULE, Lexema: %s\n", yytext); }
"strings:"            { printf("TOKEN: TK_STRINGS, Lexema: %s\n", yytext); }
"condition:"          { printf("TOKEN: TK_CONDITION, Lexema: %s\n", yytext); }
$[a-zA-Z0-9_]+        { printf("TOKEN: TK_VARIABLE, Lexema: %s\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]* { printf("TOKEN: TK_IDENTIFICADOR, Lexema: %s\n", yytext); }
\"[^\"]*"             { printf("TOKEN: TK_CADENA_LITERAL, Lexema: %s\n", yytext); }
"="                   { printf("TOKEN: TK_ASIGNACION, Lexema: %s\n", yytext); }
"{"                   { printf("TOKEN: TK_LLAVE_ABRE, Lexema: %s\n", yytext); }
"}"                   { printf("TOKEN: TK_LLAVE_CIERRA, Lexema: %s\n", yytext); }
}
[ \t\n]+              { /* Ignorar nuevas líneas, espacios y tabulaciones */ }
.                      { printf("ERROR LÉXICO: Carácter no reconocido: %s\n",
yytext); }
%%
int main(int argc, char **argv) {
    if (argc > 1) {
        FILE *file = fopen(argv[1], "r");
        if (!file) {
            perror("Error al abrir el archivo");
            return 1;
        }
        yyin = file;
    }
    yylex();
    return 0;
}
```

Conclusión

La elaboración y desarrollo de este informe ha permitido comprobar de manera empírica que el análisis léxico es un proceso altamente versátil, cuya estrategia de implementación puede y debe adaptarse a las necesidades de eficiencia, robustez y control de cada proyecto de *software*. Por un lado, la construcción manual de un lexer empleando directamente expresiones regulares —como se demostró en el caso de la tokenización de archivos Docker— otorga al desarrollador un control granular absoluto sobre el flujo de ejecución, posibilitando la integración de lógicas de recuperación ágiles que registran y obvian los errores en una sola pasada sin detener abruptamente el escaneo del archivo. Por otro lado, la adopción de herramientas estandarizadas de metacompilación como Flex, aplicada aquí para el análisis del subconjunto del lenguaje Rust, revela cómo la abstracción de la teoría de autómatas finitos optimiza de forma drástica los tiempos de desarrollo; esta técnica permite al ingeniero enfocarse exclusivamente en la definición declarativa de los patrones léxicos, delegando en la herramienta la generación automática del código en C, lo que la ratifica como el estándar indiscutible de la industria para lenguajes con gran densidad semántica.

Más allá de la tokenización puramente lineal, el estudio integral de los autómatas de pila reitera una máxima esencial de la informática teórica: las herramientas de escaneo léxico deben trabajar en ineludible sincronía con modelos de memoria más avanzados para lograr construir el árbol de derivación que requieren las gramáticas de programación modernas. Paralelamente, la reflexión aplicada a los sistemas de seguridad informática subraya que el diseño estricto y preciso de los analizadores léxicos trasciende con creces el entorno puramente académico de los compiladores tradicionales; constituye una barrera defensiva de primer nivel para prevenir errores catastróficos al procesar reglas de intrusión o firmas de *malware* en protocolos críticos como YARA. En definitiva, el dominio fluido de las expresiones regulares, la comprensión profunda de los límites de cada autómata y la correcta elección entre implementaciones artesanales o mediadas por metacompiladores representan competencias técnicas insustituibles, capacitando a los ingenieros para construir herramientas veloces, precisas y altamente resilientes en cualquier ecosistema tecnológico.

Referencias Bibliográficas

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Pearson / Addison-Wesley. Capítulo 3, "Lexical Analysis", pp. 109-189.
- Levine, J. (2009). *flex & bison: Text Processing Tools*. O'Reilly Media. Capítulos 1 ("Introducing Flex and Bison"), 2 ("Using Flex") y 5 ("A Reference for Flex Specifications").
- Márquez, F. (2026). *Tema 4: Análisis léxico* [Material de la asignatura Lenguaje y Compiladores, Sección 01]. Universidad Nacional Experimental de Guayana, Coordinación de Ingeniería en Informática. pp. 1-14.